



Faculty of Engineering
Modern Compiler Design
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester BE in
..... has satisfactorily completed his term working
(Subject name) (subject code:) during
academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASST. PROF. RADHIKA GOKANI
SUBJECT IN-CHARGE

ASST. PROF. RADHIKA GOKANI
HOD

DR.BHAVNA SONI
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1	Write a program that scans the input source code and counts the number of keywords, identifiers, operators, constants, and special symbols.			
2	Design and implement a lexical analyzer that reads source code and identifies valid tokens such as keywords, identifiers, numbers, operators, and delimiters.			
3	Write a program that simulates a Deterministic Finite Automaton (DFA) for validating whether a given string is accepted or rejected based on defined transition rules.			
4	Write a program that detects and removes left recursion from a given context-free grammar to make it suitable for top-down parsing.			
5	Write a program that performs left factoring on a grammar to eliminate ambiguity and make it compatible with predictive parsing.			
6	Write a program to compute the FIRST set of non-terminals in a given grammar, which helps in constructing parsing tables.			
7	Write a program to compute the FOLLOW set of non-terminals in a grammar, used in LL(1) parser construction.			
8	Write a program that constructs the LL(1) parsing table using FIRST and FOLLOW sets for predictive parsing.			
9	Write a program that generates Three Address Code (TAC) for arithmetic expressions to represent intermediate code.			
10	Write a program that converts intermediate code into Quadruple format for efficient code representation.			

PRACTICAL -1

AIM: Write a program that scans the input source code and counts the number of keywords, identifiers, operators, constants, and special symbols.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char word[]) {
    char *keywords[] = {
        "int", "float", "char", "double", "if", "else",
        "while", "for", "return", "void", "break", "continue"
    };

    int n = sizeof(keywords) / sizeof(keywords[0]);

    for(int i = 0; i < n; i++) {
        if(strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int isOperator(char ch) {
    char operators[] = "+-*/=%<>!";

    for(int i = 0; i < strlen(operators); i++) {
        if(ch == operators[i])
            return 1;
    }
    return 0;
}

int isSpecialSymbol(char ch) {
    char symbols[] = "(){}#";

    for(int i = 0; i < strlen(symbols); i++) {
        if(ch == symbols[i])
            return 1;
    }
    return 0;
}

int main() {
    char ch, word[50];
```

```
int i = 0;

int keywords = 0, identifiers = 0;
int operators = 0, constants = 0, symbols = 0;

FILE *fp;
fp = fopen("input.c", "r");

if(fp == NULL) {
    printf("Error opening file.\n");
    return 0;
}

while((ch = fgetc(fp)) != EOF) {

    if(isalpha(ch)) {
        word[i++] = ch;

        while(isalnum(ch = fgetc(fp))) {
            word[i++] = ch;
        }

        word[i] = '\0';
        i = 0;

        if(isKeyword(word))
            keywords++;
        else
            identifiers++;
    }

    else if(isdigit(ch)) {
        constants++;

        while(isdigit(ch = fgetc(fp)));
    }

    else if(isOperator(ch)) {
        operators++;
    }

    else if(isSpecialSymbol(ch)) {
        symbols++;
    }
}
```

```
fclose(fp);

printf("\n===== TOKEN COUNT =====\n");
printf("Keywords      : %d\n", keywords);
printf("Identifiers    : %d\n", identifiers);
printf("Operators      : %d\n", operators);
printf("Constants       : %d\n", constants);
printf("Special Symbols : %d\n", symbols);

return 0;
}
```

Input File (input.c)

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;
    int sum = a + b;
    return 0;
}
```

Output

```
===== TOKEN COUNT =====
Keywords      : 4
Identifiers    : 5
Operators      : 2
Constants       : 3
Special Symbols : 8
```

PRACTICAL -2

AIM: Design and implement a lexical analyzer that reads source code and identifies valid tokens such as keywords, identifiers, numbers, operators, and delimiters.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isKeyword(char word[]) {
    char *keywords[] = {
        "int", "float", "char", "double", "if", "else",
        "while", "for", "return", "void", "break", "continue"
    };

    int n = sizeof(keywords) / sizeof(keywords[0]);

    for(int i = 0; i < n; i++) {
        if(strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int isOperator(char ch) {
    char operators[] = "+-*/=%<>!";

    for(int i = 0; i < strlen(operators); i++) {
        if(ch == operators[i])
            return 1;
    }
    return 0;
}

int isDelimiter(char ch) {
    char delimiters[] = "({},[]";

    for(int i = 0; i < strlen(delimiters); i++) {
        if(ch == delimiters[i])
            return 1;
    }
    return 0;
}

int main() {
```

```
char ch, buffer[50];
int i = 0;

FILE *fp;
fp = fopen("input.c", "r");

if(fp == NULL) {
    printf("Error opening file.\n");
    return 0;
}

printf("\n===== LEXICAL ANALYSIS =====\n\n");

while((ch = fgetc(fp)) != EOF) {

    // (identifier / keyword)
    if(isalpha(ch)) {
        buffer[i++] = ch;

        while(isalnum(ch = fgetc(fp))) {
            buffer[i++] = ch;
        }

        buffer[i] = '\0';
        i = 0;

        if(isKeyword(buffer))
            printf("%s --> Keyword\n", buffer);
        else
            printf("%s --> Identifier\n", buffer);
    }

    else if(isdigit(ch)) {
        buffer[i++] = ch;

        while(isdigit(ch = fgetc(fp))) {
            buffer[i++] = ch;
        }

        buffer[i] = '\0';
        i = 0;

        printf("%s --> Number\n", buffer);
    }
}
```

```
        else if(isOperator(ch)) {
            printf("%c --> Operator\n", ch);
        }

        else if(isDelimiter(ch)) {
            printf("%c --> Delimiter\n", ch);
        }
    }

    fclose(fp);
    return 0;
}
```

Input File (input.c)

```
int main() {
    int a = 10;
    float b = 20;
    if(a < b) {
        a = a + 1;
    }
    return 0;
}
```

Output

===== LEXICAL ANALYSIS =====

```
int --> Keyword
main --> Identifier
( --> Delimiter
) --> Delimiter
{ --> Delimiter
int --> Keyword
a --> Identifier
= --> Operator
10 --> Number
; --> Delimiter
float --> Keyword
b --> Identifier
= --> Operator
20 --> Number
; --> Delimiter
```

if --> Keyword
(--> Delimiter
a --> Identifier
< --> Operator
b --> Identifier
) --> Delimiter
{ --> Delimiter
a --> Identifier
= --> Operator
a --> Identifier
+ --> Operator
1 --> Number
; --> Delimiter
} --> Delimiter
return --> Keyword
0 --> Number
; --> Delimiter
} --> Delimiter

PRACTICAL -3

AIM: Write a program that simulates a Deterministic Finite Automaton (DFA) for validating whether a given string is accepted or rejected based on defined transition rules.

```
#include <stdio.h>
#include <string.h>

// DFA for binary strings ending with "01"

int main() {
    char input[100];
    int state = 0; // Start state → q0

    printf("Enter binary string: ");
    scanf("%s", input);

    for(int i = 0; i < strlen(input); i++) {

        char ch = input[i];

        switch(state) {

            case 0: // q0
                if(ch == '0')
                    state = 1; // q1
                else if(ch == '1')
                    state = 0; // stay q0
                else {
                    printf("Invalid input (Not Binary)\n");
                    return 0;
                }
                break;

            case 1: // q1
                if(ch == '0')
                    state = 1; // stay q1
                else if(ch == '1')
                    state = 2; // q2
                else {
                    printf("Invalid input (Not Binary)\n");
                    return 0;
                }
                break;

            case 2: // q2 (Accept State)
                if(ch == '0')
                    state = 1; // q1
```

```
        else if(ch == '1')
            state = 0; // q0
        else {
            printf("Invalid input (Not Binary)\n");
            return 0;
        }
        break;
    }
}

// Accept State = q2
if(state == 2)
    printf("String ACCEPTED ✓\n");
else
    printf("String REJECTED ✗\n");

return 0;
}
```

Input 1

Enter binary string: 1101

Output

String ACCEPTED ✓

Input 2

Enter binary string: 1110

Output

String REJECTED ✗

Input 3

Enter binary string: 1010101

Output

String ACCEPTED ✓

PRACTICAL -4

AIM: Write a program that detects and removes left recursion from a given context-free grammar to make it suitable for top-down parsing.

```
#include <stdio.h>
#include <string.h>

int main() {
    char nonTerminal, alpha[10], beta[10];

    printf("Enter the Non-Terminal (Example: A): ");
    scanf("%c", &nonTerminal);

    printf("Enter alpha (Production A -> Aα , enter α): ");
    scanf("%s", alpha);

    printf("Enter beta (Production A -> β , enter β): ");
    scanf("%s", beta);

    printf("\nGrammar with Left Recursion:\n");
    printf("%c -> %c%s | %s\n", nonTerminal, nonTerminal, alpha, beta);

    // Eliminated Grammar
    printf("\nGrammar after Removing Left Recursion:\n");

    printf("%c -> %s%c\n", nonTerminal, beta, nonTerminal);
    printf("%c' -> %s%c' | ε\n", nonTerminal, alpha, nonTerminal);

    return 0;
}
```

Input

Enter the Non-Terminal: A

Enter alpha: a

Enter beta: b

Output

Grammar with Left Recursion:

A -> Aa | b

PRACTICAL -5

AIM: Write a program that performs left factoring on a grammar to eliminate ambiguity and make it compatible with predictive parsing.

```
#include <stdio.h>
#include <string.h>

int main() {
    char nonTerminal;
    char prod1[20], prod2[20];
    char common[20];
    int i = 0;

    printf("Enter Non-Terminal (Example: A): ");
    scanf(" %c", &nonTerminal);

    printf("Enter First Production (without A->): ");
    scanf("%s", prod1);

    printf("Enter Second Production (without A->): ");
    scanf("%s", prod2);

    // Find common prefix
    while(prod1[i] == prod2[i]) {
        common[i] = prod1[i];
        i++;
    }

    common[i] = '\0';

    printf("\nGrammar Before Left Factoring:\n");
    printf("%c -> %s | %s\n", nonTerminal, prod1, prod2);

    // Remaining parts
    printf("\nGrammar After Left Factoring:\n");

    printf("%c -> %s%c\n", nonTerminal, common, nonTerminal);
    printf("%c' -> %s | %s\n",
        nonTerminal,
        prod1 + i,
        prod2 + i);

    return 0;
}
```

Input

Enter Non-Terminal: A

Enter First Production: abcd

Enter Second Production: abef

Output

Grammar Before Left Factoring:

A $\rightarrow$ abcd | abef

Grammar After Left Factoring:

A $\rightarrow$ abA'

A' $\rightarrow$ cd | ef

PRACTICAL -6

AIM: Write a program to compute the FIRST set of non-terminals in a given grammar, which helps in constructing parsing tables.

```
#include <stdio.h>
#include <ctype.h>

char productions[10][10];
char first[10];
int count;

// Function to find FIRST
void findFirst(char c) {
    int i, j;

    // यदि terminal है → FIRST वही
    if (!isupper(c)) {
        first[count++] = c;
        return;
    }

    for (i = 0; i < count; i++)
        if (first[i] == c)
            return;

    for (i = 0; i < 10; i++) {
        if (productions[i][0] == c) {

            // Production A->a...
            if (!isupper(productions[i][2])) {
                first[count++] = productions[i][2];
            }
            else {
                findFirst(productions[i][2]);
            }
        }
    }
}

int main() {
    int n, i;
    char c;

    printf("Enter number of productions: ");
    scanf("%d", &n);
```

```
printf("Enter productions (Example: A->aB):\n");
for (i = 0; i < n; i++) {
    scanf("%s", productions[i]);
}

printf("\nEnter Non-Terminal to find FIRST: ");
scanf(" %c", &c);

count = 0;
findFirst(c);

printf("\nFIRST(%c) = { ", c);
for (i = 0; i < count; i++) {
    printf("%c ", first[i]);
}
printf("}\n");

return 0;
}
```

Input

Enter number of productions: 3
Enter productions:
A->aB
A->b
B->c

Enter Non-Terminal: A

Output

FIRST(A) = { a b }

Input

Productions:
S->Aa
A->b
A->c

Output

FIRST(S) = { b c }

PRACTICAL -7

AIM: Write a program to compute the FOLLOW set of non-terminals in a grammar, used in LL(1) parser construction.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

char productions[10][10];
char follow[10];
int count;

// Function to find FOLLOW
void findFollow(char c) {
    int i, j;

    // Start Symbol → add $
    if (productions[0][0] == c) {
        follow[count++] = '$';
    }

    for (i = 0; i < 10; i++) {
        for (j = 2; j < strlen(productions[i]); j++) {

            if (productions[i][j] == c) {

                if (!isupper(productions[i][j + 1]) &&
                    productions[i][j + 1] != '\0') {
                    follow[count++] = productions[i][j + 1];
                }

                else if (isupper(productions[i][j + 1])) {
                    follow[count++] = productions[i][j + 1];
                }

                else if (productions[i][j + 1] == '\0') {

                    if (productions[i][0] != c)
                        findFollow(productions[i][0]);
                }
            }
        }
    }
}
```

```
int main() {
    int n, i;
    char c;

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (Example: A->aB):\n");
    for (i = 0; i < n; i++) {
        scanf("%s", productions[i]);
    }

    printf("\nEnter Non-Terminal to find FOLLOW: ");
    scanf(" %c", &c);

    count = 0;
    findFollow(c);

    printf("\nFOLLOW(%c) = { ", c);
    for (i = 0; i < count; i++) {
        printf("%c ", follow[i]);
    }
    printf("}\n");

    return 0;
}
```

Input

Enter number of productions: 3

Enter productions:

S->AB

A->a

B->b

Enter Non-Terminal: A

Output

FOLLOW(A) = { B }

PRACTICAL -8

AIM: Write a program that constructs the LL(1) parsing table using FIRST and FOLLOW sets for predictive parsing.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char nonTerminals[10], terminals[10];
    char first[10][10], follow[10][10];
    char table[10][10][10];
    int nt, t, i, j;

    printf("Enter number of Non-Terminals: ");
    scanf("%d", &nt);

    printf("Enter Non-Terminals:\n");
    for(i = 0; i < nt; i++)
        scanf(" %c", &nonTerminals[i]);

    printf("\nEnter number of Terminals: ");
    scanf("%d", &t);

    printf("Enter Terminals:\n");
    for(i = 0; i < t; i++)
        scanf(" %c", &terminals[i]);

    printf("\nEnter FIRST sets:\n");
    for(i = 0; i < nt; i++) {
        printf("FIRST(%c): ", nonTerminals[i]);
        scanf("%s", first[i]);
    }

    printf("\nEnter FOLLOW sets:\n");
    for(i = 0; i < nt; i++) {
        printf("FOLLOW(%c): ", nonTerminals[i]);
        scanf("%s", follow[i]);
    }

    // Initialize table with blanks
    for(i = 0; i < nt; i++)
        for(j = 0; j < t; j++)
            strcpy(table[i][j], "-");
```

```

// Fill table (Simplified Logic)
for(i = 0; i < nt; i++) {
    for(j = 0; j < strlen(first[i]); j++) {

        if(first[i][j] != '\0') {
            for(int k = 0; k < t; k++) {
                if(first[i][j] == terminals[k]) {
                    sprintf(table[i][k], "%c->%c",
                        nonTerminals[i],
                        first[i][j]);
                }
            }
        }
    }
}

if(strchr(first[i], '\0')) {
    for(j = 0; j < strlen(follow[i]); j++) {
        for(int k = 0; k < t; k++) {
            if(follow[i][j] == terminals[k]) {
                sprintf(table[i][k], "%c->\0",
                    nonTerminals[i]);
            }
        }
    }
}
}

// Display Table
printf("\n\n==== LL(1) Parsing Table =====\n\n");

printf("    ");
for(i = 0; i < t; i++)
    printf("%c    ", terminals[i]);

printf("\n");

for(i = 0; i < nt; i++) {
    printf("%c | ", nonTerminals[i]);

    for(j = 0; j < t; j++) {
        printf("%-6s ", table[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Input

Enter number of Non-Terminals: 2

Enter Non-Terminals:

S A

Enter number of Terminals: 3

Enter Terminals:

a b c

Enter FIRST sets:

FIRST(S): ab

FIRST(A): cε

Enter FOLLOW sets:

FOLLOW(S): \$

FOLLOW(A): \$

Output

===== LL(1) Parsing Table =====

	a	b	c
S	S->a	S->b	-
A	-	-	A->c

PRACTICAL -9

AIM: Write a program that generates Three Address Code (TAC) for arithmetic expressions to represent intermediate code.

```
#include <stdio.h>
#include <string.h>

int main() {
    char expr[50];
    char op1, op2, operator;
    int temp = 1;

    printf("Enter Expression (Example: a=b+c*d): ");
    scanf("%s", expr);

    printf("\n===== THREE ADDRESS CODE =====\n");

    // Scan from right to left (simple logic)
    for(int i = strlen(expr) - 1; i >= 0; i--) {

        if(expr[i] == '+' || expr[i] == '-' ||
           expr[i] == '*' || expr[i] == '/') {

            operator = expr[i];
            op1 = expr[i - 1];
            op2 = expr[i + 1];

            printf("t%d = %c %c %c\n", temp, op1, operator, op2);

            // Replace expression with temp variable
            expr[i - 1] = 't';
            expr[i] = temp + '0'; // convert int → char
            expr[i + 1] = '\0';

            temp++;
        }
    }

    printf("%c = t%d\n", expr[0], temp - 1);

    return 0;
}
```

Input

Enter Expression: $a=b+c*d$

Output

===== THREE ADDRESS CODE =====

```
t1 = c * d
t2 = b + t1
a = t2
```

Input

Enter Expression: $x=a+b*c-d$

Output

===== THREE ADDRESS CODE =====

```
t1 = b * c
t2 = a + t1
t3 = t2 - d
x = t3
```

PRACTICAL -10

AIM: Write a program that converts intermediate code into Quadruple format for efficient code representation.

```
#include <stdio.h>

int main() {
    char op, arg1, arg2, result;

    printf("Enter TAC Instruction (Example: t1=a+b): ");
    scanf("%c%c=%c%c%c", &result, &arg1, &arg2, &op, &arg2);

    printf("\n===== QUADRUPLE REPRESENTATION =====\n");

    printf("\nOperator  Arg1  Arg2  Result\n");
    printf("-----\n");

    printf("  %c    %c    %c    %c\n", op, arg1, arg2, result);

    return 0;
}
```

Input

Enter TAC Instruction: t1=a+b

Output

===== QUADRUPLE REPRESENTATION =====

Operator Arg1 Arg2 Result

+ a b t1

Input

t2=c*d

Output

Operator Arg1 Arg2 Result

* c d t2